

# Git Project Commands

---

This project provides a number of enhancements for the way I work with git projects.

When switching branches it is typically beneficial to pull the source branch and merge it into the working branch to avoid getting stale. Typically this would be done using the following process:

```
git checkout develop
git pull
git checkout feature/ABC-123
git merge develop
```

For creating a new branch there is a similar process:

```
git checkout develop
git pull
git checkout -b feature/ABC-123
```

To solve this, I have written some scripts for convenience. For solutions to the above situations:

```
git pull-develop
# or
git pd

git feature-start 123
# or
git fs 123
```

## Installing

**Note:** The `global-setup.sh` script has not yet been created and will be a future upgrade.

~~Simply run `./src/global-setup.sh` to create the aliases in your global git config. These aliases are the same for every repository, it is repo-specific config that does the magic.~~

Once global setup is complete, we can run `./src/repo-setup.sh` to create config entries and hooks in the current working directory.

## Usage

### Branch Name Format

In the project config, you get to specify a format for each dev branch type. This format will be used in the dev branch commands outlined below.

The format has one replacement token: `{ticketid}`. This will be replaced by the ID given when the commands are called.

```
# should show something like "featurebranchformat = feature/ABC-{ticketid}"
cat .git/config | grep "featurebranchformat"

# checkout existing feature branch `feature/ABC-123`
git fc 123
```

## Pulling a Permanent Branch

To pull a branch of type "develop" or "master", simply call the relevant pull command:

```
# checks out and pulls the project's defined develop branch
git pull-develop

# pulls the project's defined master branch
git pm
```

## Interacting with Development Branches

You can perform actions on dev branches (feature, bugfix, hotfix) using the following commands. Simply replace the "feature" or "f" with the relevant branch type.

```
# create a branch (e.g. feature/ABC-123)
git featurestart 123

# checkout an existing branch
git fc 123

# branch _from_ an existing branch (e.g. from feature/ABC-123 to
feature/ABC-456)
git feature-duplicate 123 456
git fd 456 123

# rename an existing branch
git feature-rename 123 456
git fr 456 123
```

**Note:** You currently cannot duplicate or rename from one branch type to another. A rename will *always* be of the same type, just a different ticket ID. For example: `git fs 123` will create a new *feature* branch. Calling `git fr 123 456` will rename to a new *feature* branch.

## Hooks

Hooks in the `./hooks` directory are automatically symlinked into a configured git repository. Other hooks can be installed manually.

### Future State Possibility

In future it may be worth allowing user hooks to be added and automatically loaded from "root" hooks.

Maybe look for e.g.

- `./hooks/pre-commit.d/*.sh`
- `~/.config/git-pc/hooks/pre-commit.d/*.sh?`

## Developing Hooks

### Globals and Functions

Hooks cannot load `globals.sh`, since the location of the *called* hook is within the repository, and the location of this project cannot be known.

It could be set up that this project's location is injected into the git config for the project, which would allow calls to the globals files from hooks. I wouldn't recommend this, however, as the hooks should *probably* stand on their own; allowing users access to the functions and scripts in this project feels like a bad idea...